

**Universidad Nacional de Ingeniería
Facultad de Ciencias**

Arquitectura de computadores

Programación en MIPS

Prof.: Lic. César Martín Cruz S.
ccruz@uni.edu.pe

2012 - II

Programación en MIPS

- Dos simuladores para Windows:
 - MARS. Desarrollado en la Universidad de Missouri.
<http://courses.missouristate.edu/KenVollmar/MARS/>
 - SPIM. Desarrollado en la Universidad de Wisconsin.
<http://www.cs.wisc.edu/~larus/spim.html>
- Ambos sistemas ofrecen:
 - Interface gráfica de usuario.
 - Ensamblador extendido con pseudo-instrucciones.
 - Simulador de una arquitectura MIPS.

MARS(*MIPS Assembler and Runtime Simulator*)

MARS es un software simulador para el lenguaje ensamblador de MIPS para uso educativo.

- MARS tiene algunas ventajas:
 - Configurable.
 - Herramientas. Por ejemplo para visualizar los accesos a memoria y un simulador de un caché de datos.
 - No requiere instalación (privilegios de administrador).

Uso de la memoria

- Por convención, los sistemas basados en MIPS dividen la memoria en 3 segmentos:
 1. Segmento de texto (*text segment*).
 2. Segmento de datos (*data segment*).
 3. Segmento de pila (*stack segment*).

Segmento de Texto

1. El segmento de texto, a partir de la dirección 400000_{16} , es donde se guarda el código del programa. En los programas, el segmento de texto se marca por medio de la instrucción **.text**.

Segmento de Datos

2. El segmento de datos (*data segment*), a partir de la dirección 10000000_{16} , es donde se guardan los datos. Se indica por medio de la instrucción **.data**.

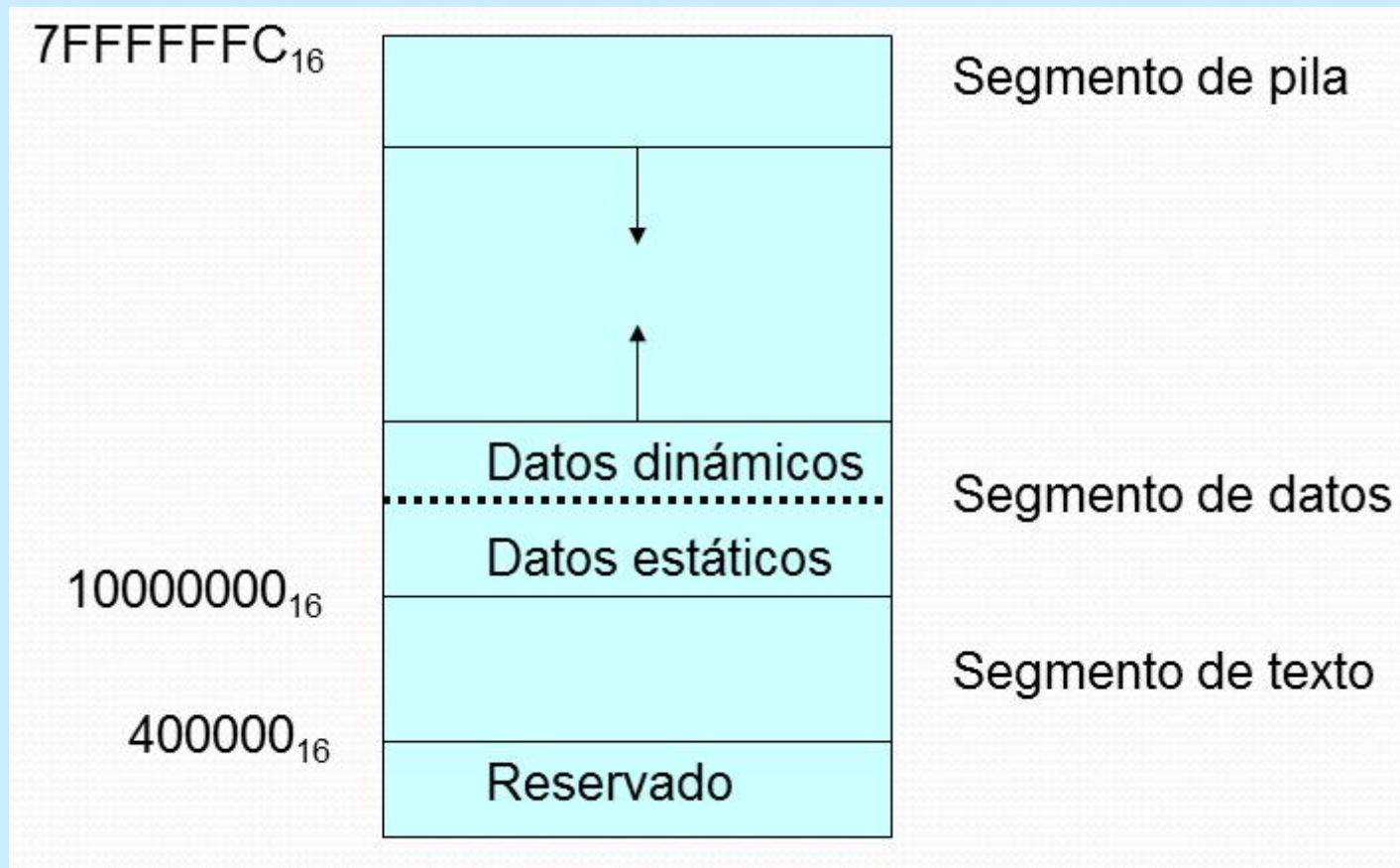
A su vez, el segmento de datos consta de dos partes:

- a) **Datos estáticos.** Contiene variables de tamaño fijo y que necesitan ser accedidos durante todo el programa. Por ejemplo, variables globales.
- b) **Datos dinámicos.** Contiene variables que se crean durante el programa.

Segmento de Pila

3. El segmento de pila (*stack segment*), a partir de la dirección 7FFFFFFC_{16} , es donde se guardan los stack frames.

Uso de la memoria



Acceso a la memoria

- Por convención, los sistemas MIPS dedican el registro **\$gp** como apuntador a los datos estáticos.
- Asignándole a **\$gp** la dirección 10008000_{16} y usando constantes de 16 bits con signo, el programa puede acceder los primeros 64KB del segmento de datos (direcciones 10000000_{16} a 10010000_{16}).
- Por ejemplo, la instrucción:
 lw \$v0, 0x8000(\$gp)
Carga la palabra que comienza en 10010000_{16} .

Llamadas a Sistema

- **SPIM** y **MARS** ofrecen servicios tipo sistema operativo a través de la instrucción *syscall*.
- Los servicios permiten leer del teclado, escribir a la consola y manejar archivos del sistema.
- Para pedir un servicio, el programa carga el número de servicio en **\$v0** y los argumentos en **\$a0-\$a3**. Los servicios que regresan un valor lo hacen en **\$v0**.

Prog. “Hola mundo” en MIPS

.data

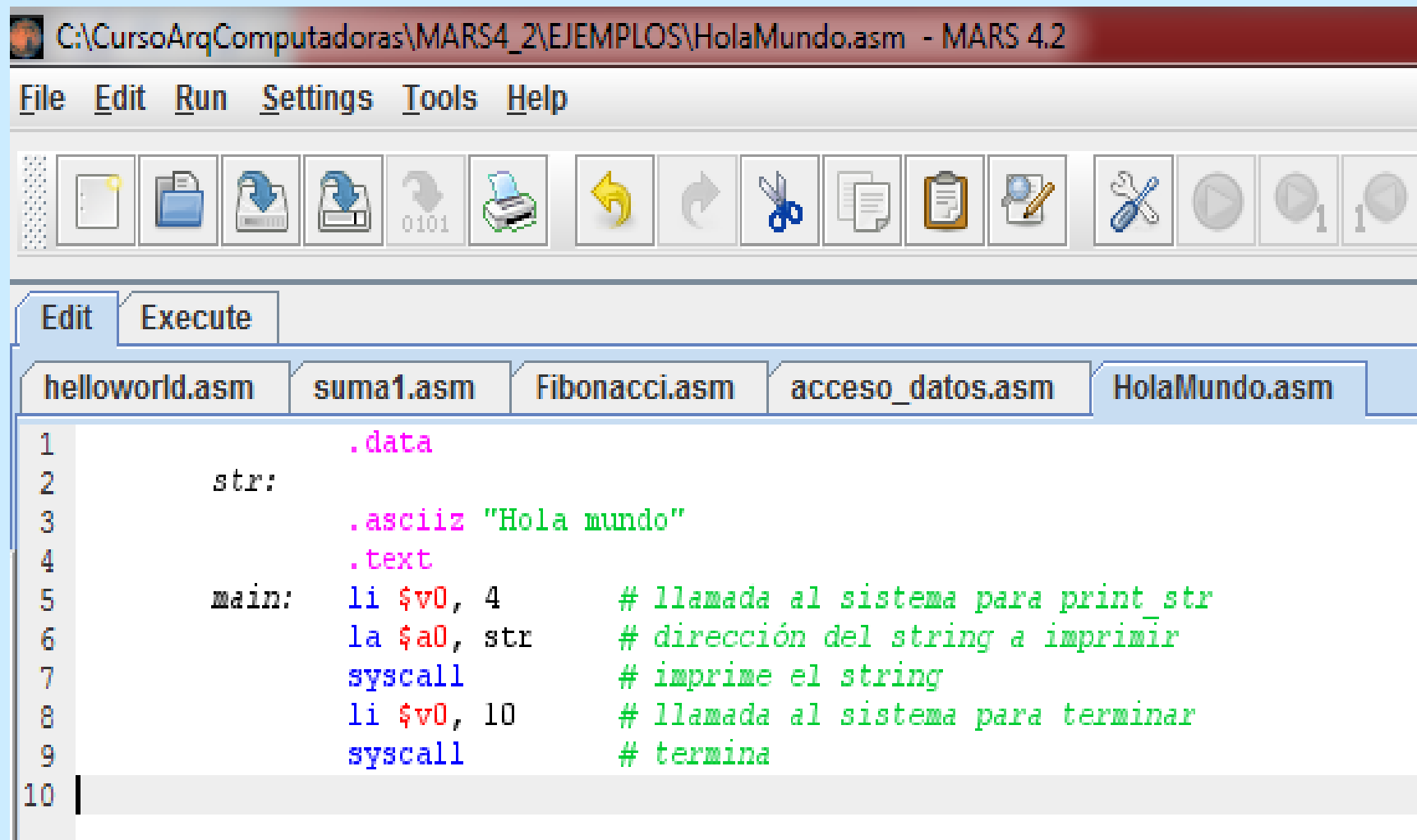
str:

.asciiz “Hola mundo”

.text

```
main: li $v0, 4      # llamada al sistema para print_str
      la $a0, str    # dirección del string a imprimir
      syscall        # imprime el string
      li $v0, 10     # llamada al sistema para terminar
      syscall        # termina
```

En Mars



```
1      .data
2      str:
3      .asciiz "Hola mundo"
4      .text
5      main: li $v0, 4      # llamada al sistema para print_str
6            la $a0, str    # dirección del string a imprimir
7            syscall        # imprime el string
8            li $v0, 10     # llamada al sistema para terminar
9            syscall        # termina
10
```

Llamadas al sistema

Servicio	Código	Argumentos	Resultado
print_int	1	sao = integer	
print_float	2	sfiz = float	
print_double	3	sfiz = float	
print_string	4	sao = string	
read_int	5		integer (en \$vo)
read_float	6		float (en \$vo)
read_double	7		double (en \$vo)
read_string	8	sao = buffer, \$a1 = tamaño	
sbrk	9	sao = cantidad	dirección (en \$vo)
exit	10		
print_char	11	sao = char	
read_char	12		char (en \$vo)
open	13	sao = nombre del archivo (string), \$a1 = banderas, \$a2 = modo	handle (en \$ao)
read	14	sao = handle, \$a1 = buffer, \$a2 = tamaño	num. de caracteres leídos (en \$ao)
write	15	sao = handle, \$a1 = buffer, \$a2 = tamaño	num. de caracteres escritos (en \$ao)
close	16	sao = handle	
exit2	17	sao = result	

Lenguaje Ensamblador

- Los números están en base 10 por default. La base 16 se indica con 0x.
- Los opcodes(código de operación) son palabras reservadas.
- Las etiquetas terminan en dos puntos (:).
- Los strings se encierran con comillas dobles (“”).
- Caracteres especiales en strings como en C
 - Nueva línea \n
 - Tabulador \t
 - Comilla \“

Directivas del Ensamblador

- **.align n** – Alinea el dato a 2^n bytes. Con $n=0$ se suspende la alineación hasta el siguiente **.data**.
- **.ascii str** – Almacena un string en memoria, pero no lo termina en nulo.
- **.asciiz str** - Almacena un string en memoria y lo termina en nulo.
- **.byte $b_1 \dots b_n$** – Almacena los valores en bytes consecutivos de memoria.
- **.data [dir]** – Define el comienzo del segmento de datos. Si el argumento opcional **dir** está presente, el segmento comienza en esa dirección.
- **.double $d_1, \dots, d_n$** – Almacena los valores reales de doble precisión en localidades consecutivas de memoria.

Directivas del Ensamblador

- **.float $f_1, \dots, f_n$** – Almacena los valores reales de precisión sencilla en localidades consecutivas de memoria.
- **.globl sym** – Declara que sym es global y puede ser accesado desde otros archivos.
- **.space n** – Reserva n bytes de espacio en el segmento actual. En SPIM debe ser el segmento de datos.
- **.text [dir]** – Define el comienzo del segmento de código. Si el argumento opcional *dir* está presente, el segmento comienza en esa dirección.
- **.word $w_1, \dots, w_n$** – Almacena los datos de 32 bits en palabras consecutivas de memoria.

Directivas del Ensamblador

- **.half h1,..., hn** – Almacena los datos de 16 bits en medias palabras consecutivas de memoria.
- **.extern sym size** – Declara que el dato almacenado en sym tiene tamaño size y es global. El dato está guardado en el segmento de datos.